

Portafolio de Evidencias — Primer Avance

Programación Orientada a Objetos

Estudiante: Daniel Quirós

Curso: POO

Docente: Romario Salas Cerdas

Fecha: 25 de octubre de 2026

1. Justificación y contexto del sistema

Sistema elegido: Buscaminas (juego clásico de lógica).
Justificación: El Buscaminas es un sistema ideal para modelar con POO porque sus componentes naturales (tablero, celdas, jugador, configuración) se mapean directamente a clases, cada celda tiene un estado interno protegido y un comportamiento especializado según su tipo (mina o número), y las reglas del juego se expresan como interacciones entre objetos bien delimitados.
Contexto: El sistema se ejecuta en consola en este primer avance, pero está diseñado para que la capa de presentación sea reemplazable por una interfaz gráfica en JavaFX sin modificar el modelo. El jugador configura el tablero (filas, columnas, minas), revela celdas y marca banderas hasta descubrir todas las celdas seguras (victoria) o pisar una mina (derrota).

2. Diseño orientado a objetos

2.1 Entidades principales

Entidad	Responsabilidad
Celda (abstracta)	Define el contrato común de toda celda: posición, estado de revelado, bandera y comportamiento polimórfico.
CeldaMina	Especialización: representa una mina, no dispara cascada, se revela como *.
CeldaNumero	Especialización: almacena minas adyacentes, dispara cascada si es 0, se revela como texto numérico.
Tablero	Administra la matriz de celdas, coloca minas, calcula adyacentes, revela con cascada y expone consultas para la vista.
Jugador	Representa al actor de la partida (identidad inmutable).
ConfiguracionJuego	Objeto de valor inmutable que agrupa y valida los parámetros iniciales.
Juego	Controla el ciclo de vida de la partida: inicio, turnos, victoria y derrota.
Coordenada	Objeto de valor inmutable que representa una posición (fila, columna).
GeneradorPosiciones	Produce coordenadas aleatorias dentro de los límites del tablero.

EstadoCelda (enum)	Vocabulario compartido entre modelo y vista: OCULTA, REVELADA, BANDERA.

2.2 Diagrama UML

[Ver documento en github]

2.3 Interpretación del UML

- Herencia (CeldaMina → Celda, CeldaNumero → Celda): las subclases heredan la estructura base y especializan el comportamiento polimórfico (esMina, requiereRevelarVecinas, valorRevelado).
- Composición (Tablero *— Celda, Juego *— Tablero): el dueño crea y controla el ciclo de vida de los componentes; si el dueño se destruye, los componentes también.
- Agregación (Juego o— Jugador): el jugador existe independientemente de la partida.
- Dependencia (Tablero > EstadoCelda, Juego > ConfiguracionJuego, Juego > GeneradorPosiciones): uso temporal o como tipo de retorno, sin posesión.

3. Implementación

3.1 Abstracción

La abstracción en este sistema se aplicó para modelar las entidades clave ocultando los detalles de implementación irrelevantes y exponiendo solo el comportamiento esencial. La clase Celda fue diseñada como una clase abstracta, ya que el concepto de una "celda genérica" no tiene existencia propia en el juego; solo existen celdas concretas (minas o números). Esta clase define el contrato común de posición y estado, y declara métodos abstractos como esMina() y valorRevelado(). Las clases CeldaMina y CeldaNumero heredan de esta base y especializan su comportamiento. Gracias a esto, el Tablero puede interactuar con cualquier celda de manera uniforme mediante polimorfismo, sin necesidad de conocer su tipo específico en tiempo de ejecución, lo que simplifica la lógica del juego y reduce la duplicación de código.

3.2 Encapsulamiento

El encapsulamiento se implementó de forma rigurosa para proteger el estado interno de los objetos y garantizar la integridad de las reglas del negocio. En la clase Celda, todos los

atributos de estado (fila, columna, estaRevelada, tieneBandera) son estrictamente privados. Las subclases no acceden a ellos directamente, sino a través de métodos de consulta marcados como final, lo que impide que las clases hijas alteren la lógica fundamental de revelado o marcado de banderas. De igual manera, la clase Tablero oculta por completo su matriz interna de celdas. En lugar de exponer la estructura de datos, ofrece métodos de consulta (obtenerEstado y obtenerValor) que devuelven tipos de datos simples (un enumerado y un texto). Esto asegura que la capa de presentación (ya sea consola o una futura interfaz gráfica) solo pueda leer la información necesaria sin tener la capacidad de mutar el modelo.

3.3 Relaciones entre objetos

Las relaciones entre los objetos se definieron aplicando el principio de distribución de responsabilidades, asegurando que cada clase se especialice en una tarea única y que el acoplamiento se mantenga al mínimo.

Se estableció una relación de composición entre Tablero y Celda, así como entre Juego y Tablero. En estos casos, el objeto dueño controla el ciclo de vida de sus componentes; el tablero crea y gestiona la matriz de celdas, y estas no pueden existir de manera independiente fuera de él.

Por otro lado, la relación entre Juego y Jugador se modeló como una agregación. El jugador es una entidad que existe antes de que inicie la partida y persiste después de que esta termina; el juego simplemente utiliza su identidad para registrar los resultados, pero no es dueño de su ciclo de vida.

Finalmente, se utilizaron relaciones de dependencia para interacciones temporales o de uso puntual. Por ejemplo, Juego depende de ConfiguracionJuego para recibir los parámetros validados en su constructor, y Tablero depende de GeneradorPosiciones y Coordenada únicamente durante la fase de inicialización para colocar las minas. Mantener estas relaciones como dependencias en lugar de asociaciones permanentes reduce el acoplamiento y hace que el sistema sea más flexible y fácil de mantener

4. Reflexión personal

El desarrollo del Buscaminas me enfrentó a desafíos técnicos y conceptuales que pusieron a prueba mi comprensión de la POO más allá de la teoría.

Desafíos técnicos. La lógica de métodos como inicializarTablero() y revelarCelda() fue compleja de implementar, especialmente por el uso de matrices bidimensionales. Manejar índices, validar límites y coordinar la recursividad de la cascada de revelado requirió depuración constante. Aprendí que una matriz no es solo una estructura de datos, sino el esqueleto del modelo: si se expone mal, se rompe el encapsulamiento; si se recorre mal, se rompe la lógica del juego. Por eso opté por que Tablero nunca devolviera la matriz ni sus celdas, sino solo datos primitivos (EstadoCelda, String) a través de obtenerEstado() y obtenerValor().

Desafíos conceptuales. A diferencia de sistemas del mundo real (una biblioteca, una agenda), el Buscaminas es un dominio abstracto: no hay un "objeto mina" tangible que uno pueda señalar en la vida cotidiana. Modelar entidades como Celda, CeldaMina, CeldaNumero o EstadoCelda exigió un ejercicio de abstracción más deliberado, porque no podía apoyarme en analogías inmediatas. Tuve que preguntarme en cada clase: *¿cuál es su responsabilidad única? ¿Qué sabe y qué no debe saber?* Esa pregunta fue la que me permitió distinguir composición de agregación: Tablero *compone* celdas (las crea y las controla), pero Juego *agrega* un Jugador (que existe antes y después de la partida).